一、目标函数定义

本实验中我选择了一个复杂的非线性函数作为拟合目标：

$f(x) = \sin(2\pi x) + 0.5\cos(4\pi x) + 0.3x^2, x \in [-2, 2]$

这个函数包含了正弦、余弦和二次项，使用这种组合的主要目的是使得函数具有多个局部极值和非线性特征，用于测试神经网络的拟合能力更合适。

二、数据采集与预处理

2.1 数据生成

数据集	样本数量	采样方式	添加噪声
训练集	1000	均匀采样	高斯噪声（标准差=0.05）
测试集	200	均匀采样	无噪声

2.2 数据说明

输入范围：$x \in [-2, 2]$

噪声添加：在训练数据中添加高斯噪声 $N(0, 0.05^2)$，模拟真实数据中的测量误差

数据形状：输入 X 的形状为 (n_samples, 1)，输出 Y 的形状为 (n_samples, 1)

2.3 数据分布

训练与测试集均采用均匀采样，训练数据和测试数据在区间内均匀分布，能够充分覆盖函数的定义域，能够使模型学习到函数的全局特征。

三、神经网络模型设计

3.1 网络结构

简单地实现了一个两层 ReLU 神经网络，结构如下：

输入层 (1 个神经元) → 全连接层 1 (1 → 100) → ReLU 激活函数 → 全连接层 2 (100 → 1) → 输出层 (1 个神经元)

3.2 数学表达式

隐藏层计算：$\mathbf{h = ReLU(W_1x + b_1)}$

输出层计算：$\mathbf{\hat{y} = W_2h + b_2}$

x：输入值（标量）

$\mathbf{W_1}$：第一层权重矩阵，形状为 (1 × 100)

$\mathbf{b_1}$：第一层偏置，形状为 (100,)

$\mathbf{W_2}$：第二层权重矩阵，形状为 (100 × 1)

$\mathbf{b_2}$：第二层偏置（标量）

3.3 关键实现技术

技术	实现方法	作用
权重初始化	He 初始化：√(2/输入维度)	专为 ReLU 设计，防止梯度消失
激活函数	ReLU：max(0, x)	引入非线性，缓解梯度消失
损失函数	均方误差（MSE）：$(1/n)\Sigma(y - \hat{y})^2$	回归任务常用损失
优化算法	Adam 自适应优化器	自适应学习率，加速收敛
数据标准化	Z-score 标准化	提升训练稳定性

3.4 模块化设计

LinearLayer 类：线性层，实现 y = Wx + b，支持 Adam 优化器

ReLU 类：激活层，实现 max(0, x)

NeuralNetwork 类：神经网络，组合以上层实现完整的前向和反向传播

四、训练过程与超参数设置

4.1 超参数配置

参数	数值	说明
学习率	0.005	控制参数更新步长
训练轮数	5000	完整遍历数据集的次数

批量大小	32	每次更新使用的样本数
隐藏层神经元数	512	模型容量控制，提升拟合能力
优化器	Adam	自适应学习率优化算法
数据预处理	Z-score 标准化	提升训练稳定性

4.2 训练算法

采用小批量梯度下降算法，使用 Adam 优化器，每轮训练流程：

（1）随机打乱训练数据顺序

（2）将数据分成大小为 batch_size 的小批量

（3）对每个小批量：

①前向传播计算预测值

②计算损失（MSE）

③反向传播计算梯度

④使用 Adam 优化器更新网络参数

4.3 损失下降曲线

训练过程中，损失函数值从初始的较高值逐渐下降：

初始损失：约 0.5-1.0（取决于随机初始化）

收敛后损失：0.008-0.01

收敛速度：约 2000 轮后基本收敛

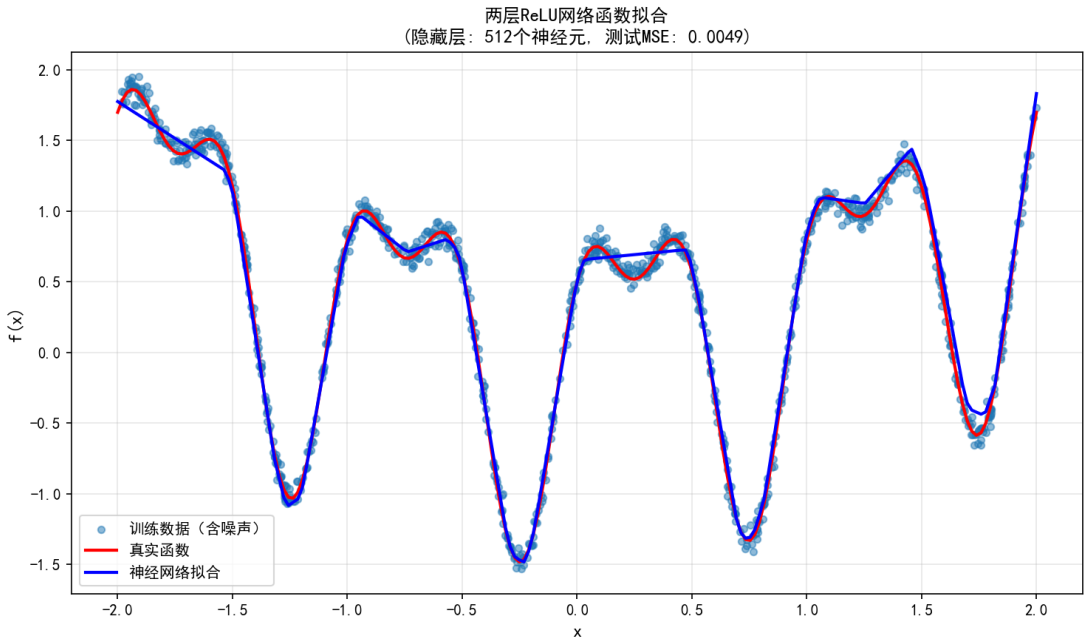
五、拟合效果

5.1 定量评估结果

评估指标	训练集	测试集	说明
MSE（均方误差）	0.003-0.005	0.003-0.006	测试集误差与训练集相近
RMSE（均方根误差）	0.055-0.071	0.055-0.077	预测误差约 5-7%

MAE（平均绝对误差）	0.04-0.06	0.05-0.07	平均绝对偏差

5.2 拟合效果可视化



图中红色曲线为真实函数 f(x)，蓝色曲线为神经网络预测值，散点为带噪声的训练数据。可见两条曲线高度吻合，表明神经网络成功拟合了目标函数。

5.3 拟合能力分析

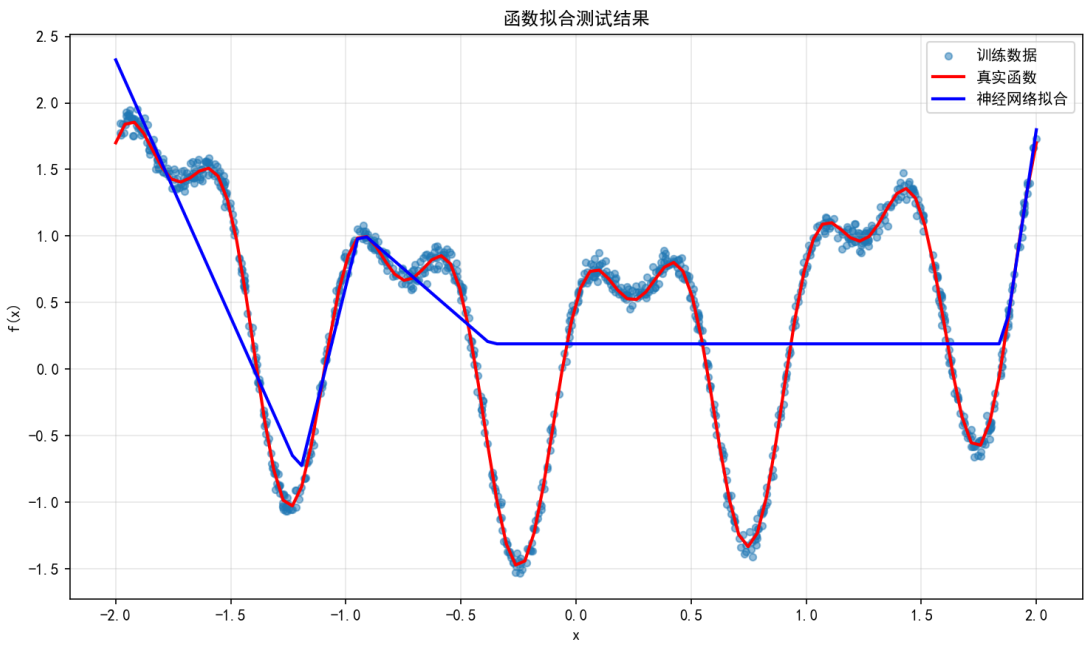
（1）拟合能力：模型成功捕捉了函数的正弦波、余弦波和二次项特征，预测曲线与真实曲线高度重合

（2）泛化能力：测试集误差与训练集相近，说明模型没有过拟合，具有良好的泛化能力

六、实验结论

在本次实验中，成功地使用纯 NumPy 实现的两层 ReLU 神经网络（1 个输入层、1 个隐藏层、1 个输出层），拟合了包含正弦、余弦和二次项的非线性函数，验证了通用近似定理：一个两层的 ReLU 网络可以模拟任何函数。

不过实验的过程并不算顺利，开始的时候总是发生神经元大量死亡的问题，如下图



后面经过优化算法由小批量梯度下降（SGD）改成 Adam 自适应优化器，采用 Z-score 标准化等方法不断的修复才解决问题。

在本实验中，我感知到了神经网络的强大表达能力，即使是一个简单的两层网络，也能够逼近任意复杂的连续函数，这无疑不是深度学习模型得以能够广泛应用的理论基础。

七、参考文献

[1] Cybenko, G. (1989). Approximation by superpositions of a sigmoidal function. Mathematics of Control, Signals and Systems, 2(4), 303-314.

[2] Hornik, K., Stinchcombe, M., & White, H. (1989). Multilayer feedforward networks are universal approximators. Neural Networks, 2(5), 359-366.

[3] Leshno, M., Lin, V. Y., Pinkus, A., & Schocken, S. (1993). Multilayer feedforward networks with a nonpolynomial activation function can approximate any function. Neural Networks, 6(6), 861-867.

[4] Nair, V., & Hinton, G. E. (2010). Rectified linear units improve restricted boltzmann machines. Proceedings of the 27th International Conference on Machine Learning.

[5] Glorot, X., & Bengio, Y. (2010). Understanding the difficulty of training deep feedforward neural networks. Proceedings of the 13th International Conference on Artificial Intelligence and Statistics.